

LoadLead - SMS Opt-In Program

Platform Engineering Review with Courses of Action

Verdict: STAGED GO | 2026-07-02 | source: docs/sms-program-review.md (commit dcdeb27) | prepared by the platform team

LoadLead SMS Program - Platform Review

Platform Engineering review of the proposed build: carrier SMS opt-in, provider-neutral messaging layer (AWS End User Messaging default, Telnyx and Plivo stubs), append-only consent record, two public SMS legal pages, and the A2P 10DLC operator runbook (Parts A through L). Same method as docs/telematics-integration-review.md: read-only recon, judgments grounded in the actual repo with file paths, honest unknowns, courses of action (COAs) with explanations at every real decision point, and a prioritized roadmap.

Nothing here was implemented or applied. This document is the only output.

1. Executive summary

- **The platform is unusually well prepared for this build.** The three hardest things the prompt worries about already exist as proven patterns: an append-only ESIGN consent store that captures version, hash, server timestamp, IP, and user agent (the accessorial policy acceptances); a fail-closed integrations adapter framework with test-mode capture and deploy-time stub pruning (services/integrations); and correct client-IP handling behind the proxy (trust proxy is already set, and consent-grade rows already record req.ip and the user agent).
- **The prompt mis-states the stack in two places and both matter:** it says Postgres (the platform is DynamoDB only, no pg dependency, no RDS, no SQL migration tooling) and it says the SES integration is the model to follow (the live email adapter is Resend; SES SDK packages exist as dependencies but the shipping path is services/integrations/email.ts on Resend). The consent store must be a DynamoDB table and the messaging adapter should follow the Resend adapter's shape, not a hypothetical SES one.
- **Two of the prompt's assumptions do not match the frontend:** there is no shared Footer component and no separate personas page (persona cards live on the Landing page, frontend-v2/src/pages/Landing.tsx). And the mandated dark NAVY palette with Cambria/Calibri directly conflicts with the repo's white-glass design system. COA-3 resolves this.
- **The critical path is not code.** A2P 10DLC brand plus campaign approval plus number association is realistically 2 to 6 weeks of vendor-side waiting. The code (roughly 2 weeks) fits entirely inside that window. The sequencing consequence: publish the two legal pages and start Part A registration immediately; everything else can follow.

- **One cross-team hard requirement the prompt does not know about:** the compliance layer suppresses routine notifications for loads under lawful non-disclosure orders (`PushService.send` consults `LawEnforcementService.isEntityRestricted`). SMS load alerts are exactly the class of message that check exists for. The SMS send path must honor the same suppression seam or it becomes a legal-hole bypass. COA-5.
 - **Headline recommendation: staged go.** Start the operator registration track today, build the legal pages and consent record first (they gate campaign approval), land the messaging adapter inside the existing integrations framework, and wire broadcast last, behind the suppression seam and the sandbox-tested loop.
-

2. Reality check: prompt assumptions vs the actual platform

Prompt says	Repo says	Consequence
Stack includes Postgres; enforce immutability with REVOKE or triggers; migrations directory	DynamoDB only. No pg in backend/package.json, no RDS in infra/terraform, no SQL migrations. Tables are provisioned via infra/terraform + backend/scripts/createTables.mjs + a key in backend/src/config/environment.ts	Consent store is a DynamoDB table; immutability via the platform's own tiers (COA-2), not SQL grants
"Already using SES"; SES integration is the model	Live email path is a Resend adapter (backend/src/services/integrations/email.ts). AWS SDK v3 is present for other services, so adding @aws-sdk/client-pinpoint-sms-voice-v2 is consistent	Follow the integrations adapter conventions (modeResolver, bootGuard, captureStore, stub pruning), not an SES pattern that is not the live one
Home page and a separate personas page with shared Footer	Landing.tsx is the public home and carries the five persona cards (persona work item #100); there is no personas page file and no shared Footer component in frontend-v2/src/components	Footer links land on Landing (and the extraction of a small shared Footer is the cleaner path); the "personas page footer" requirement collapses into the same surface
Capture IP behind CloudFront/ALB from the forwarded header ("gotcha")	Already solved: app.set('trust proxy', 1) at backend/src/index.ts:64, and the ESIGN acceptance rows already record req.ip + user agent (backend/src/routes/accessorials.ts:185-186)	Reuse as-is; nothing to invent
"Any existing immutable consent module... extend its pattern"	It exists and is nearly isomorphic: AccessorialPolicyAcceptance rows carry version, sha256 policy hash, attestation text, server signedAt, ipAddress, userAgent, append-only (backend/src/services/accessorialPolicyService.ts)	The consent store is a sibling of this store, not a new invention. The disclosure-hash discipline (server recomputes, never trusts client text) is already the house style
Brand palette NAVY/CARD/INK dark theme, Cambria/Calibri	White-glass design system: customer-glass.css / admin-glass.css, design tokens per design-system/MASTER.md, PageHeader conventions	Direct conflict between two of the prompt's own global constraints ("use exactly this palette" vs "match the repo's styling system"). COA-3
Legal pages publicly reachable without the beta login	Compatible today: the beta wall gates sign-in on the apex (frontend-v2/src/lib/host.ts, PrivateBetaLanding.tsx), not all routes; SPA fallback serves 200 for any route (verified pattern)	Public Routes in App.tsx work; carrier reviewers get real pages
Five-state carrier verification machine (UNVERIFIED,	Verification statuses live in backend/src/services/verification.ts; consent-independent gating matches the platform's existing separation (verify gates hauling, not consent)	The prompt's "do not couple consent to verification" already matches house philosophy

Prompt says	Repo says	Consequence
PENDING, VERIFIED, REJECTED, EXPIRED)		
Carrier phone number already collected	Yes: signup profile carries phone (backend/src/services/authService.ts:53,97; User/profile types backend/src/types/index.ts)	Opt-in UI can prefill

3. Infrastructure inventory (grounded)

Area	Finding	Source
Runtime/deploy	Node 22 EB WebServer tier only; deploy-backend.sh is the only path to prod; APP_ENV-locked, stub-pruning scan	infra/terraform/envs/prod/eb-imported.tf:51-52, deploy-backend.sh
Integrations framework	Adapter-per-provider with modeResolver + bootGuard (prod fail-closed), captureStore + /_test outbox for test mode, stubs pruned from deploy zips	backend/src/services/integrations/ (README.md), email.ts, push.ts
Webhook/event ingress	Proven: raw-body-before-json mount, HMAC over exact bytes, idempotent ingest, inert-when-unconfigured (Tally); rawBody verify hook exists globally for Didit	backend/src/index.ts:159-171, routes/tallyWebhook.ts, services/tallySignature.ts
SNS ingress	Does NOT exist yet. Current inbound is vendor HTTPS webhooks. SNS HTTP subscriptions add two new requirements: the SubscriptionConfirmation handshake and SNS message signature verification (X509 cert chain), which is a different trust model than shared-secret HMAC	new work; pattern must be built once and reused for both topics
Consent precedent	Append-only ESIGN acceptances with version + sha256 + server time + IP + UA; append-only trust events; append-only stop events; WORM S3 sink for signatures via Lambda with CloudWatch alarms	backend/src/services/accessorialPolicyService.ts, betaTrustEventService.ts, stopEventService.ts, infra/terraform/envs/prod/worm-sink.tf
Client IP / UA	trust proxy set; req.ip + user-agent captured on consent-grade rows today	backend/src/index.ts:64, routes/accessorials.ts:185-186
Broadcast engine	BroadcastService drives offer rebroadcast via in-process setInterval (flagged in-repo as debt); user-facing notify chokepoint is PushService.send, which enforces the non-disclosure suppression seam	backend/src/index.ts:247-250, services/broadcastService.ts, services/pushService.ts:60-67
Notification suppression seam	PushService.send suppresses when LawEnforcementService.isEntityRestricted says the lead is under a non-disclosure order. SMS must honor the same check	backend/src/services/pushService.ts, docs/TeamOwnership.md RACI
Secrets	EB env vars only; no Secrets Manager/KMS in app code (same finding as the telematics review; SMS does not strictly need KMS since the AWS provider uses the instance role, but Telnyx/Plivo API keys later would land as env vars per convention)	deploy-backend.sh, config/environment.ts

Area	Finding	Source
Observability	Console logger -> CloudWatch; SNS ops-alerts topic + alarm pattern exists; topic still has zero subscribers	backend/src/utils/logger.ts, infra/terraform/envs/prod/observability.tf:24-26
Frontend	Vite + React SPA; public pages exist (Landing, PrivateBetaLanding); no shared Footer; white-glass tokens; VITE_* build-time env	frontend-v2/src/pages/Landing.tsx, src/lib/host.ts, .env.production
IAM	EB instance role managed in terraform; least-privilege addition (sms-voice:SendMessage scoped to the number ARN) fits the existing pattern	infra/terraform/envs/prod/

4. Courses of action (the decision points)

COA-1: Where the consent record lives

COA	Description	Pros	Cons
1A (recommended)	New dedicated DynamoDB table LoadLead_SmsConsentEvents, append-only, modeled on the AccessorialPolicyAcceptance row shape (id PK; userId/carrierId by reference; phoneE164; channel; consentAction OPT_IN/OPT_OUT/DECLINED; consentMethod; source; disclosureVersion; disclosureSha256 server-computed; consentedAt server-set; ipAddress; userAgent)	Follows the house pattern exactly (dedicated single-purpose store, reference by id, append-only); provisioning is the established terraform + createTables + environment.ts triple; schemaless store tolerates method/source extension	One more table (trivial); scan-based reads until a phone GSI is added for the inbound-STOP lookup
1B	Reuse/extend the existing AccessorialPolicyAcceptances store with a new record kind	Zero new tables	Pollutes a load-scoped legal store with a user-scoped program consent; different retention and legal-hold profiles; settlements owns that store. Rejected
1C	Postgres per the prompt (new RDS + migrations + REVOKE/triggers)	Matches the prompt text	Introduces an entire database platform, migration tooling, VPC wiring, and ops burden for one table; contradicts every convention in the repo. Rejected outright, same verdict as the telematics review's Postgres stores

Explanation: the prompt itself instructs "follow the existing LoadLead data pattern: dedicated, single-purpose stores that reference other entities by id" and "if an immutable consent module exists, extend its pattern." Both point at 1A. The TCPA-defense properties the prompt cares about (exact-text fidelity via version + server-side sha256, server timestamp, IP, UA) are byte-for-byte the properties the acceptance store already demonstrates.

COA-2: How immutability is enforced

COA	Description	Pros	Cons
2A (baseline, required)	App-layer append-only convention (no update/delete code paths) + PITR + deletion protection on the table, as done for all 21 recent prod tables	Zero new machinery; matches every append-only store in the platform	Immutability is conventional, not mechanical; a buggy or malicious code path could still UpdateItem
2B (recommended addition)	2A plus an explicit IAM Deny for dynamodb:UpdateItem and dynamodb:DeleteItem on the consent table ARN, attached to the EB instance role in terraform	Mechanical enforcement, the DynamoDB equivalent of the prompt's REVOKE UPDATE, DELETE; cheap (a terraform statement); auditor-friendly answer to "prove rows cannot be mutated"	First table with a per-table deny; must remember the deny when writing future migrations/backfills (a deliberate speed bump, which is the point)
2C (gold, later)	2B plus mirroring consent events into the existing S3 WORM sink pipeline (the signatures pattern, worm-sink.tf)	Compliance-grade, off-host, object-lock immutability; strongest litigation posture	Extends a Lambda pipeline for modest marginal benefit at beta volume; do when SMS volume or counsel demands it

Explanation: the prompt demands database-level immutability "where the repo allows it." The repo allows 2B today with one terraform statement. 2A alone matches current convention but leaves the auditor question open; 2C is the roadmap answer, not the launch answer.

COA-3: Legal pages surface and styling

COA	Description	Pros	Cons
3A (recommended)	Public SPA routes /sms-privacy-policy and /sms-terms in App.tsx, styled with the repo's white-glass design system; exact legal COPY preserved byte for byte; extract a small shared Footer used by Landing and the new pages	One product, one design language; reviewer reachability verified by the SPA-fallback pattern already proven in prod; the copy (which is what carriers review) is untouched	Deviates from the prompt's NAVY palette mandate
3B	Implement the prompt's dark NAVY/Cambria palette scoped to the two legal pages only	Follows the prompt literally	Two visual identities on one public site; carrier reviewers do not require any particular palette, only reachability, exact disclosure copy, and the no-share clause; permanent inconsistency for zero compliance gain
3C	Static pre-rendered HTML on S3/CloudFront outside the SPA	Loads without JS; maximally robust for automated reviewers	Splits deploy pipelines and drifts from the app footer/header; the A2P review process is human-with-a-browser, so the robustness benefit is small

Explanation: the prompt contains two mutually exclusive global constraints (exact palette vs match the repo's styling). Compliance reviewers verify reachability and copy, not colors. Resolve in favor of the platform's design system and treat the palette section of the prompt as intended for a repo this one is not. If the palette is actually a hard brand requirement from the business, 3B is the fallback; flag it as a product decision, not an engineering one.

COA-4: Where the messaging abstraction lives

COA	Description	Pros	Cons
4A (recommended)	Build MessagingProvider inside services/integrations as a new adapter family: aws-sms.ts (live), telnyx.ts + plivo.ts as stubs in the stubs/ directory, factory keyed by MESSAGING_PROVIDER through the existing modeResolver conventions, test mode writing to captureStore (SMS outbox visible at /_test/outbox), bootGuard registering the new integration so a non-live mode refuses to boot in production	The swap-by-env-var requirement is exactly what modeResolver already does for FMCSA/EMAIL/MAPS/PUSH/DIDIT; stubs get pruned from prod zips automatically by deploy-backend.sh's scan; test mode comes for free; one convention instead of two	The prompt's standalone "messaging module" shape must be adapted (superficial)
4B	Standalone messaging/ module per the prompt, outside the integrations framework	Matches prompt structure	Re-implements mode selection, stubbing, test capture, and prod guards that already exist; two integration conventions forever. Rejected

Explanation: the prompt's stated goal (start on AWS, swap to Telnyx/Plivo by flipping one env var, no caller changes) is the exact design brief the integrations framework was built to satisfy. providerAutoHandlesOptOut, parseInbound, parseStatus fit as interface members regardless of home. The one prompt requirement to keep verbatim: no caller outside the adapter ever imports a concrete provider.

COA-5: The SMS send path and the suppression seam

COA	Description	Pros	Cons
5A (recommended, required)	All outbound SMS goes through one chokepoint (an smsService or an extended notification dispatcher) that (1) consults the consent resolver, (2) consults LawEnforcementService.isEntityRestricted for load-scoped messages exactly as PushService.send does, then (3) calls the provider interface	Consent gating and legal suppression enforced in one place; matches the platform's single-chokepoint philosophy (pushService); compliance team's seam stays intact	Slightly more design than "call sendSms from the broadcaster"
5B	Broadcast engine calls the provider factory directly, consent check inline	Less code	Recreates the notification-suppression hole the compliance layer just closed; every future SMS call site must remember two checks. Rejected

Explanation: this is the finding the prompt could not have known. A load under a lawful non-disclosure order must not generate a text message to a carrier when it would not generate a push. The suppression check lives at the notify chokepoint today (pushService.ts); SMS is a second door into the same room and must have the same lock. Compliance is Consulted on this seam per docs/TeamOwnership.md.

COA-6: Sequencing against the A2P long pole

COA	Description	Pros	Cons
6A (recommended)	Two parallel tracks. Ops track starts today: publish legal pages first (they gate campaign review), then runbook Parts A, C, D in order with E through I during the waits. Code track proceeds meanwhile: consent store, adapter, opt-in UI, broadcast wiring, sandbox/simulator testing	Registration wait (2 to 6 weeks) fully absorbs the build (about 2 weeks); go-live is gated by the Campaign Registry, not by engineering	Requires the legal pages plus an opt-in screenshot early, so Phase 3/6 UI comes before some backend polish
6B	Build everything first, register after	Simple ordering	Adds the full registration wait to the calendar after code is done; the opt-in screenshot and live legal URLs are needed for Part C anyway. Rejected

Explanation: the runbook itself says Part A is the long pole. The only engineering artifacts the registration needs early are the two public pages and a screenshot of the opt-in screen. Front-load exactly those.

5. Per-phase assessment

Phase	Feasibility	Effort	Notes
P1 Recon	done	-	This document is that recon; all requested paths reported in section 3 (router frontend-v2/src/App.tsx; layout frontend-v2/src/layouts/AppLayout.tsx; home Landing.tsx; personas = Landing persona section; onboarding Signup.tsx + carrier flow; consent precedent accessorialPolicyService.ts; AWS client config backend/src/config/aws.ts; env example backend/.env.staging pattern; "migrations" = createTables.mjs + terraform)
P2 Messaging abstraction	NOW	M (3-5 d)	Per COA-4A inside services/integrations; new dep @aws-sdk/client-pinpoint-sms-voice-v2 (SDK v3 house-standard); SNS ingress endpoint is new work: SubscriptionConfirmation handshake + SNS signature verification + idempotent processing, one implementation shared by the inbound and events topics
P3 Legal pages	NOW, first	S (1-2 d)	Per COA-3A; exact copy byte for byte including the no-share callout; both routes public; effective date constant
P4 Footer links	NOW	S (0.5-1 d)	Extract shared Footer; wire on Landing (home + personas requirement both land there) and the opt-in screen
P5 Consent record	NOW	M (3-4 d)	Per COA-1A + COA-2B; disclosure constant + server-side sha256 mirrors the acceptance-store discipline; opt-in endpoint follows house route conventions with express-validator; resolver = newest event per (user, channel), the same newest-non-superseded read used by stopEventService.effectivePair; confirmation SMS through the chokepoint
P6 Opt-in UI	NOW	S-M (2-3 d)	CarrierSmsConsent in onboarding; unchecked by default; DECLINED written when proceeding without opting in; prefill phone from the signup profile; no localStorage; a11y per repo standards
P7 Tests/DoD	NOW	S (1-2 d)	Mirror the route-test harness (supertest + signed JWTs + mocked services: tests/unit/payments/factoringRoutes.test.ts is the template); factory/provider-swap tests mirror the existing per-adapter mode tests
Runbook A-D (brand, campaign, number)	OPS, start today	days of forms, weeks of waiting	Needs EIN-exact legal details, domain email, live legal URLs, opt-in screenshot. Note: prompt sets legal-page effective date June 30, 2026 which is in the past relative to today (2026-07-02); set the real publish date
Runbook E-F (config set + two SNS topics)	OPS + terraform	S	Provision via terraform (aws_sns_topic x2, configuration set), not console-only, so prod stays declared; app env gets both topic ARNs
Runbook G-H (spend cap, sandbox exit)	OPS	S	Enforced spend limit low (\$20) before any real send; production-access support case with the same copy as Part C
Runbook I (IAM)	NOW (terraform)	S	sms-voice:SendMessage on the EB instance role, Resource tightened to the number ARN once known; receiving needs no app-side IAM

Phase	Feasibility	Effort	Notes
Runbook J-L (env, test loop, go-live)	after code + registration	S	Simulator-number end-to-end loop before real sends; counsel review is an explicit go-live gate

6. Prioritized roadmap

NOW (week 1)

1. **Legal pages + shared Footer** (P3+P4, COA-3A). They gate campaign registration and cost a day or two.
2. **Start runbook Part A (brand registration)** the same day the pages are live. Ops task, needs EIN-exact company data.
3. **Consent store + disclosure constant + opt-in endpoint** (P5, COA-1A/2B).
4. **Opt-in UI in onboarding** (P6) - produces the screenshot Part C needs.

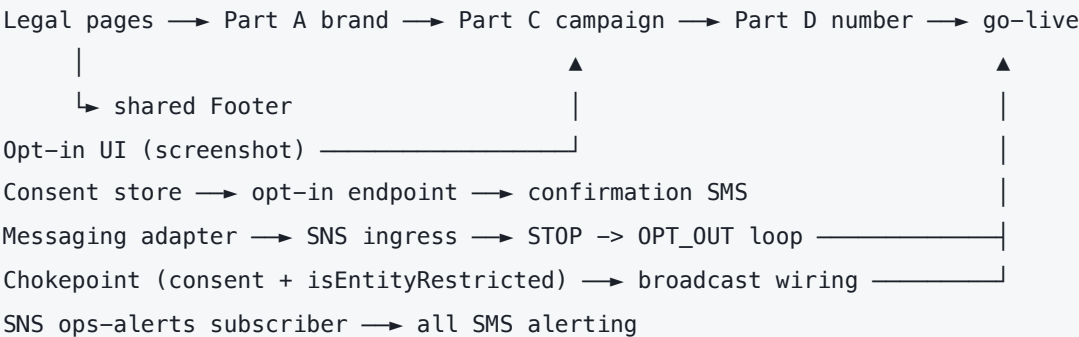
NEXT (weeks 2-3, overlapping the registration wait)

5. **Messaging adapter family + factory + stubs** (P2, COA-4A).
6. **SNS ingress endpoint** (confirmation handshake, signature verification, idempotency) + terraform for the two topics and the configuration set.
7. **SMS chokepoint honoring consent + suppression seam** (COA-5A) and broadcast wiring behind it.
8. **Runbook C/D** as approvals arrive; E through I in the gaps; tests (P7).

LATER (gated)

9. **Simulator loop, sandbox exit, spend cap, counsel sign-off, go-live** (runbook J-L) - gated on campaign approval + number association.
10. **WORM mirror for consent events** (COA-2C) when volume or counsel asks.
11. **Telnyx/Plivo real implementations + voice** only if AWS End User Messaging proves limiting.

Dependency graph



Team routing (for the eventual Jira conversion; not applied)

Workstream	Team	Consulted
Messaging adapter, SNS ingress, terraform (topics, config set, IAM, spend cap), env wiring	platform	-
Consent store + opt-in endpoint + resolver	identity (onboarding/consent of a user)	compliance (legal record posture), platform (store provisioning)
Opt-in UI + legal pages + footer	marketplace (public surface, design system)	identity
Broadcast SMS wiring behind the chokepoint	marketplace	compliance (suppression seam), platform
A2P runbook execution, counsel review	ops/owner	compliance

7. Risks and open questions

1. **A2P timing variance** (days to ~6 weeks for campaign, ~2 weeks for number association). Mitigated by COA-6A sequencing; cannot be engineered away.
2. **Message-type discipline.** The campaign registers as Transactional. Every template must stay operational in tone; a later marketing blast through this campaign risks carrier suspension. Recommend a template registry constant next to the disclosure constant so copy is reviewable in one place.
3. **SNS signature verification** is new code with real security weight (an unauthenticated public endpoint that mutates consent state on STOP). Must verify the SNS message signature and the TopicArn allowlist, and be idempotent by provider message id. Do NOT replicate the Didit test-event signature bypass (backend/src/services/verification.ts:370-375), the same anti-pattern flagged in the telematics review.
4. **STOP scope semantics:** an inbound STOP arrives with only a phone number. If two users ever share a number, the OPT_OUT must apply to the number, not one user. The resolver should key

opt-out reads by phoneE164

- channel with userId as attribution. Design detail, decide before the store schema freezes.

5. **Quiet hours / time-of-day sending:** TCPA safe-harbor practice limits send windows (commonly 8am to 9pm recipient local time). Load alerts are time-sensitive and transactional, but this is a counsel question; the chokepoint should leave a seam for a send-window policy either way.
 6. **Effective-date literal:** the prompt hardcodes June 30, 2026, already in the past. Use the actual publish date; keep it a constant.
 7. **The prompt's palette vs the design system** is a product decision if anyone insists on NAVY; engineering default is COA-3A.
 8. **Personas page does not exist** as a separate page; the requirement is satisfied on Landing. If a standalone personas page is planned, the shared Footer from P4 makes the link placement automatic.
 9. **Broadcast worker debt:** SMS broadcast will ride the same in-process rebroadcast interval the repo already flags as debt (index.ts:247-250) until the telematics review's scheduled-worker enabler lands. Not a blocker for opt-in; is a scale risk for high-volume alerting. The two initiatives share that enabler.
 10. **Unknowns that need the vendor console, not the repo:** current AWS account sandbox status, existing origination identities, whether the account already has any End User Messaging configuration. Verify in console before Part A.
-

8. Recommendation

Staged go.

- Go now on the legal pages, shared footer, consent store (DynamoDB, COA-1A + IAM-deny immutability, COA-2B), opt-in UI, and the ops registration track in parallel (COA-6A).
- Build the messaging layer inside the existing integrations framework (COA-4A) with the SNS ingress done carefully (signature verification, idempotency, no test bypasses).
- Wire broadcast last, through a single chokepoint that enforces consent AND the compliance suppression seam (COA-5A) - this is the one requirement the prompt missed and the platform must not.
- Reject the Postgres store and the SES-as-model assumptions; note the palette conflict as a product decision defaulting to the house design system.
- Go-live remains gated on: campaign approved, number associated, sandbox exited, spend cap set, simulator loop green, counsel sign-off on the legal copy.